

08_DISTRIBUTED SYSTEMS

1. INTRODUZIONE

Distributed system (sistemi distribuiti): a distributed system is a set of spatially separated entities, each of them with a certain computational power, that are able to communicate and to coordinate among themselves for reaching a common goal and that appears to its users as a single coherent system. A distributed system is a collection of independent computers that appear to the users of the system as a single system.

Ad esempio: Network of workstations, Distributed manufacturing system (e.g., automated assembly line), Network of branch office computers.

Distributed systems are useful for availability of powerful yet cheap microprocessors (PCs, workstations), continuing advances in communication technology.

Esempi di networked applications associati a tipi di applicazione:

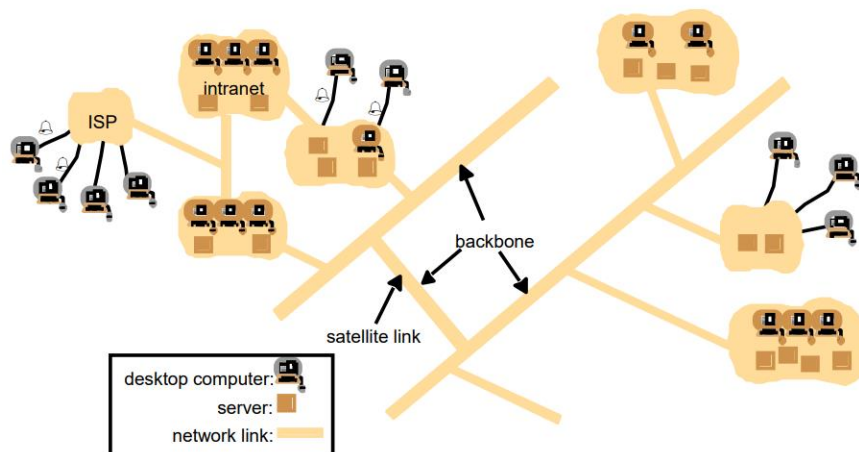
<i>Finance and commerce</i>	eCommerce e.g. Amazon and eBay, PayPal, online banking and trading
<i>The information society</i>	Web information and search engines, ebooks, Wikipedia; social networking: Facebook and MySpace.
<i>Creative industries and entertainment</i>	online gaming, music and film in the home, user-generated content, e.g. YouTube, Flickr
<i>Healthcare</i>	health informatics, on online patient records, monitoring patients
<i>Education</i>	e-learning, virtual learning environments; distance learning
<i>Transport and logistics</i>	GPS in route finding systems, map services: Google Maps, Google Earth
<i>Science</i>	The Grid as an enabling technology for collaboration between scientists
<i>Environmental management</i>	sensor technology to monitor earthquakes, floods or tsunamis

Esempi di distributed systems:

- Local Area Network and Intranet
- Database Management System
- Automatic Teller Machine Network
- Internet/World-Wide Web
- Pervasive Systems and Ubiquitous Computing
- Service Oriented Architecture
- Virtual networks
- Peer-to-peer (P2P)
- Cloud Computing
- Big Data Computing

creati per rispondere alle esigenze dei domini applicativi.

Internet: sistema distribuito dove più macchine consultano dati nei server passando attraverso gli ISP



World-Wide-Web: applicazione più grande al mondo. Collezione di tutte le pagine che qualcuno ha creato e messo a disposizione di altri.

Massively multiplayer online games (MMOGs): offer an immersive experience whereby very large numbers of users interact through the Internet with a persistent virtual world.

Include: complex playing arenas (l'utente deve interagire col mondo nel videogioco e con altri utenti. Le interazioni che faccio col mondo e con altri utenti devono essere propagate real time a tutti gli utenti del gioco), social systems (voglio poter comunicare con altri giocatori), financial systems (conversione di soldi reali con soldi digitali)

Require: fast responses, real time propagation of events, consistent view of the entire world

Vantaggi dell'uso di Distributed Systems rispetto a Centralized Systems:

- Economics: a collection of microprocessors offer a better price/performance than mainframes. Low price/performance ratio: cost effective way to increase computing power.
Un unico server potente costa tanto. Posso comprare tante macchine più economiche raggiungendo la stessa potenza.
- Speed: a distributed system may have more total computing power than a mainframe.
For example: 10,000 CPU chips, each running at 50 MIPS. Not possible to build 500,000 MIPS single processor since it would require 0.002 nsec instruction cycle. Enhanced performance through load distributing.
Un unico server non avrà più di tanti core, anche se potenti. Raggiungiamo quindi dei limiti di velocità dati dalla potenza dei core e dal numero limitato di essi.
- Inherent distribution: Some applications are inherently distributed.
For example a supermarket chain.
- Reliability: If one machine crashes, the system as a whole can still survive. Higher availability and improved reliability.
Se ho un unico server e mi crush, allora devo sistemarlo prima di poter continuare.
- Incremental growth: Computing power can be added in small increments. Modular expandability.
- Another deriving force: the existence of large number of personal computers and IoTs, the need for people to collaborate and share information.

Vantaggi dell'uso di Distributed Systems rispetto a Independent PCs:

- Data sharing: allow many users to access to a common data base.
- Resource Sharing: expensive peripherals like color printers.
- Communication: enhance human-to-human communication (for example email, chat).
- Flexibility: spread the workload over the available machines

Svantaggi dell'uso di Distributed Systems:

- Software: difficult to develop software for distributed systems
- Network: saturation, lossy transmissions
- Security: easy access also applies to secrete data

Primary goal dei distributed systems: sharing data/resources

Problemi: sincronizzazione e coordinazione

Coordination: si deve tener conto delle seguenti condizioni, che non si presentavano nel caso di centralized systems:

- Temporal and spatial concurrency.
Si deve gestire la concorrenza spaziale, che prima non si verificava: il mio dato potrebbe essere salvato tramite repliche in diverse macchine allo stesso tempo.
- No global Clock. Non c'è un clock globale per tutti, possono variare da macchina a macchina.
- Failures. Le macchine potrebbero crashare, ad esempio se sono in attesa di un dato da una macchina che per crasha, che faccio?
Ci vuole qualcuno che riesca a capire quando una macchina crasha.
- Unpredictable latencies.

These limitations restrict the set of coordination problems we can solve in a distributed setting.

Quando si fa il design di un Distributed System si deve gestire i seguenti aspetti:

- Heterogeneity
- Openness, come riesco a fornire servizi fra le varie componenti
- Security
- Scalability, nuove macchine devono poter essere facilmente integrate nel mio sistema e vecchie macchine devono poter uscire semplicemente
- Reliability
- Concurrency, nell'accesso alle risorse
- Flexibility
- Performance
- Transparency

Vediamo gli aspetti sovrastanti:

- **Heterogeneity:**
 - Networks, il sistema distribuito si appoggia su diversi tipi di rete
 - computer hardware, la mia applicazione deve poter girare su macchine composte da dispositivi hardware diversi (su mac, windows, android, ...)
 - operating systems, deve funzionare su diversi OS
 - programming languages, diversi linguaggi di programmazione

- implementations by different developers
- mobile code

- **Openness:** distributed services may be usable. Their interfaces must be published.

Un sistema distribuito aperto è basato su un meccanismo di comunicazione uniforme e delle interfacce pubbliche, per l'accesso a shared resources.

Open distributed systems can be constructed from heterogeneous hardware and software. The conformance of each component to the published standard must be carefully tested.

Ogni elemento del sistema distribuito deve essere testato secondo uno standard.

- **Security:** dobbiamo garantire:

- **confidentiality:** impedire l'accesso a chi non è autorizzato alle risorse. Protection against disclosure to unauthorized individuals
- **integrity:** protection against alteration or corruption. Il dato gestito dal sistema non deve essere alterato (per errore, come durante la trasmissione, o da altri non autorizzati) o corrotto.
- **availability:** protection against interference with the means to access the resources.

- **Scalability:** systems grow with time or become obsolete (la crescita non è lineare).

Problemi da gestire:

- Controlling the cost of physical resources.
- Controlling the performance loss. Io in un distributed system inserisco un overhead per la gestione delle macchine. Questo non dev'essere predominante.
- Preventing software resources running out.
- Avoiding performance bottlenecks. Esempi di bottleneck: centralized components (a single mail server), centralized tables (a single URL address book), centralized algorithms (routing based on complete information)

- **Reliability:** distributed system should be more reliable than single system.

Ad esempio 3 machines with 95% probability of being up. $(95\%)^3$ probability of being up.

Si devono poter gestire i failures nel seguente modo:

- Detecting failures
- Masking failures
- Tolerating failures
- Fault recovery
- Redundancy

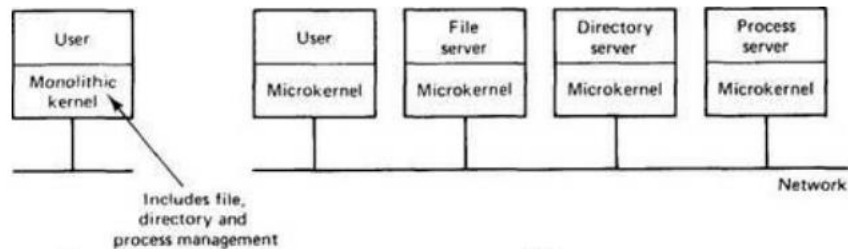
- **Concurrency:** temporal and spatial concurrency.

- **Flexibility:** il sistema si deve adattare rapidamente a qualsiasi cambiamento.

Make it easier to change.

Si devono usare meno **Monolithic Kernel**: systems calls are trapped and executed by the kernel. All system calls are served by the kernel (for example UNIX).

Usare più **Microkernel**: provides minimal services (InterProcess Communication, some memory management, some low-level process management and scheduling, low-level I/O).



- **Performance:**

- Performance loss due to communication delays: si deve fare meno fine-grain parallelism, tante piccole task parallele comportano una grande comunicazione per riunire il risultato di queste task (high degree of interaction); e si deve fare più coarse-grain parallelism.
- Performance loss due to making the system fault tolerant.

- **Transparency:** voglio far sembrare che sia tutto in un'unica macchina.

How to achieve the single-system image, i.e., how to make a collection of computers appears as a single computer.

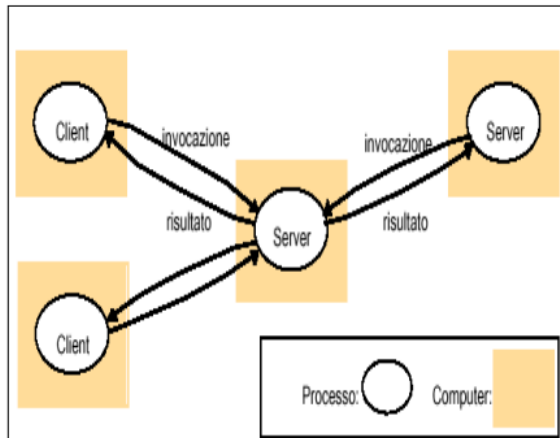
- **Access Transparency:** local and remote resources are accessed using the same operations
- **Location Transparency:** users cannot tell where hardware and software resources such as CPUs, printers, files, data bases are located.
- **Concurrency Transparency:** the users are not aware of the existence of other users. Need to allow multiple users to concurrently access the same resource. Lock and unlock for mutual exclusion.
- **Replication Transparency:** OS can make additional copies of files and resources without users noticing.

Diversi tipi di trasparenza:

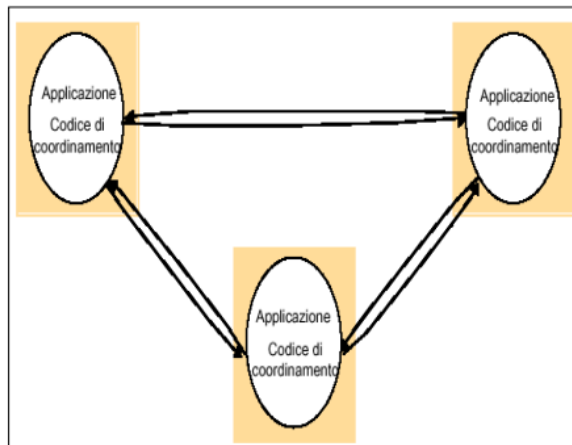
- **Failure transparency:** enables the concealment of faults, allowing users and application programs to complete their tasks despite the failure of hardware or software components.
- **Migration Transparency:** resources and clients must be free to move from one location to another without affecting the operations or requiring their names changed (ad esempio /usr/lee, /central/usr/lee).
- **Performance transparency:** allows the system to be reconfigured to improve performance as loads vary.
- **Scaling transparency:** allows the system and applications to expand in scale without change to the system structure or the application algorithms.
- **Parallelism Transparency:** Automatic use of parallelism without having to program explicitly. The holy grail for distributed and parallel system designers.

Modelli di interazione:

- Client/Server:



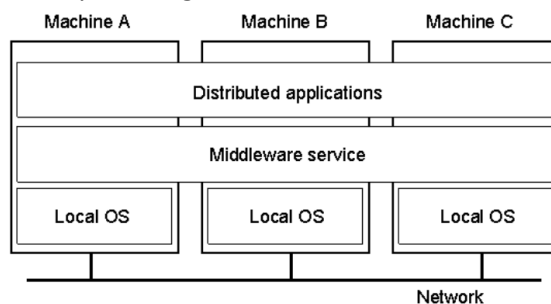
- Peer-to-Peer:



Layering hardware e software (su un'unica macchina):



A distributed system organized as middleware. The middleware layer extends over multiple machines.



Quando chiedo l'elaborazione di un certo dato può quindi risiedere in una qualunque delle macchine.

Problemi nello sviluppo del middleware:

- **Heterogeneity:** OS, clock speeds, data representation, memory, architecture HW. Tutte caratteristiche che possono essere differenti sulle varie macchine.
- **Local Asynchrony:** troppo carico di lavoro su un nodo, different HW, Interrupts.
- **Lack of global knowledge:** knowledge propagates through messages whose propagation time will be much slower than the time taken by the execution of an internal event
- **Network Asynchrony:** propagation times of message can be unpredictable
- **Failures** of nodes or network partitions
- **Lack of a global order of events**
- Consistency vs Availability vs Network Partitions

These problems limit the set of problems that can be solved through deterministic algorithms on some distributed systems.

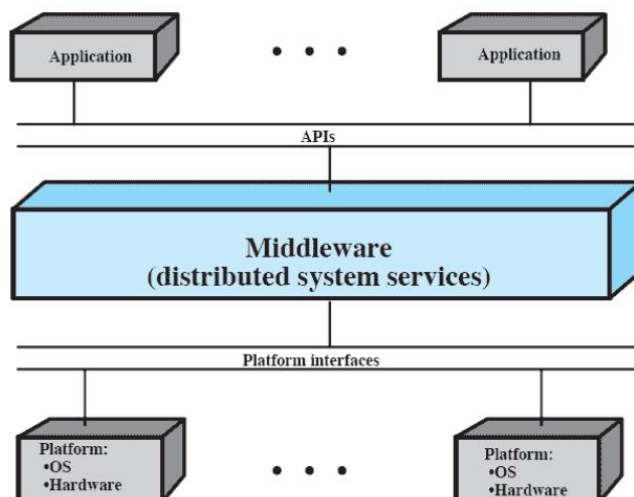
To achieve the true benefits of the distributed system, developers must have a set of tools that provide a uniform means and style of access to system resources across all platforms.

This would enable programmers to build applications that look and feel the same.

Enable programmers to use the same method to access data regardless of the location of that data.

The way to meet this requirement is by the use of standard programming interfaces and protocols that sit between the application (above) and communications software and operating system (below).

Logical View of Middleware:



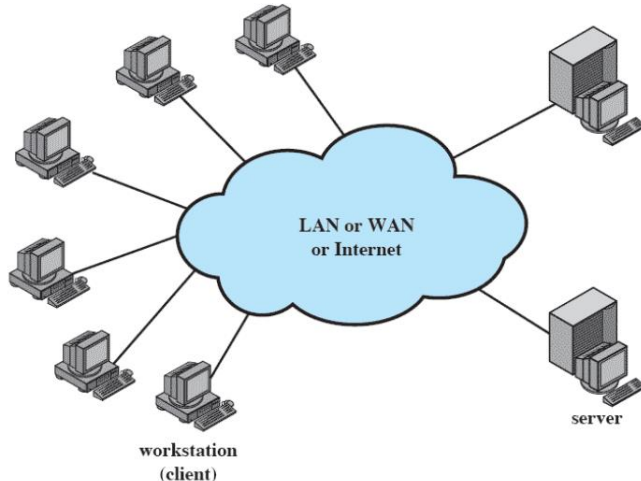
2. CLIENT/SERVER COMPUTING

Client machines are generally single-user PCs or workstations that provide a highly user-friendly interface to the end user.

Each server provides a set of shared services to the clients.

The server enables many clients to share access to the same database and enables the use of a high-performance computer system to manage the database.

Generic Client/Server environment:



Caratteristiche di client/server (a differenza di altri tipi di distributed processing):

- there is a heavy reliance on bringing user-friendly applications to the user on his or her own system
- there is an emphasis on centralizing corporate databases and many network management and utility functions
- there is a commitment, both by user organizations and vendors, to open and modular systems
- networking is fundamental to the operation

Client/Server applications: bulk of applications software executes on the server. Application logic is located at the client. Presentation services in the client.

The key feature of a client/server architecture is the allocation of application level tasks between clients and servers.

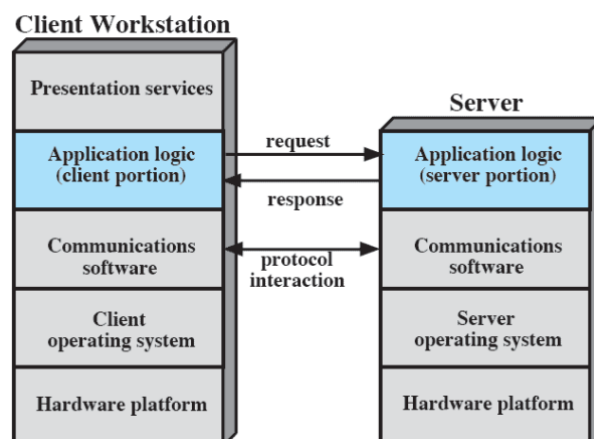
Hardware and the operating systems of client and server may differ.

These lower-level differences are irrelevant as long as a client and server share the same communications protocols and support the same applications.

It is the communications software that enables client and server to interoperate (principalmente TCP/IP). Actual functions performed by the application can be split up between client and server in a way that optimizes the use of resources.

The design of the user interface on the client machine is critical: there is heavy emphasis on providing a graphical user interface (GUI) that is easy to use, easy to learn, yet powerful and flexible.

Generica architettura client/server:



Business-oriented applications (payroll, order entry, customer tracking, inventory control, etc.) contain four general components:

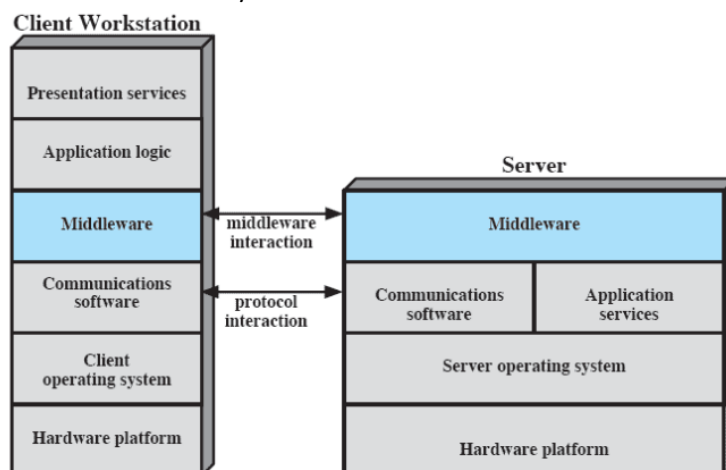
- **Presentation logic:** user interface.
- **I/O processing logic:** data validation.
- **Business processing logic:** business rules and calculations.
- **Data storage logic:** constraints such as primary keys, referential integrity, and actual data retrieval.

While almost all applications contain those four general components, for any given application those components need not be part of the same program, resident on the same computer, written in the same language, nor written by the same group of programmers.

In application component development bisogna porsi le seguenti domande:

- What language should a component be written in?
- What hardware resource should a component reside upon?
- How often will the component change? (language changes, platform changes, business changes).
- Who is responsible for maintaining the component?
- How long is the application supposed to last?

Role of Middleware in Client/Server architecture:



Nel server gli application services stanno sotto il middleware, in questo modo posso comunicare direttamente con l'OS. È più efficiente.

Ci sono 4 classi di applicazioni client/server:

- **Host-based processing**
Not true client/server computing, traditional mainframe environment.



- **Server-based processing**

Server does all the processing, client provides a graphical user interface.



- **Client-based processing**

All application processing done at the client, data validation routines and other database logic functions are done at the server.



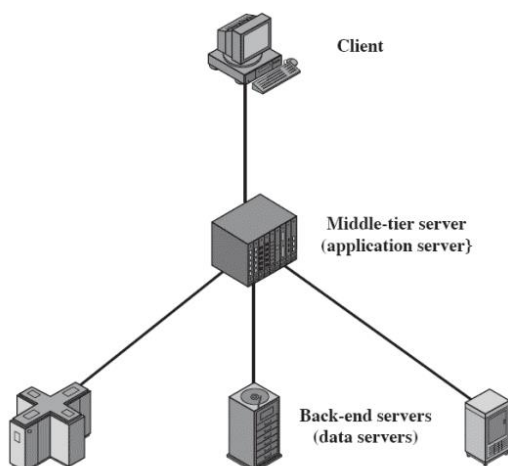
- **Cooperative processing**

Application processing is performed in an optimized fashion. Complex to set up and maintain, Fat vs Thin clients.



Three-tier Client/Server Architecture (a 3 livelli): application software distributed among three types of machines:

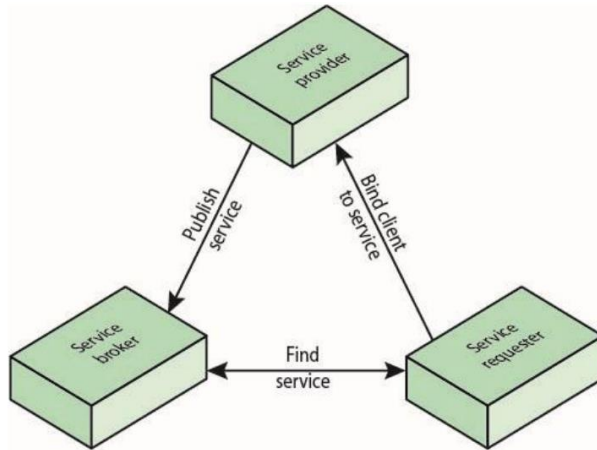
- **User machine** (Thin client)
- **Middle-tier server**: server dedicato allo smistamento dei lavori (gateway, convert protocols, merge/integrate results from different data sources). Se chiedo un certo servizio il middle-tier server sa a quale server backend riferirsi per il servizio.
- **Backend server**



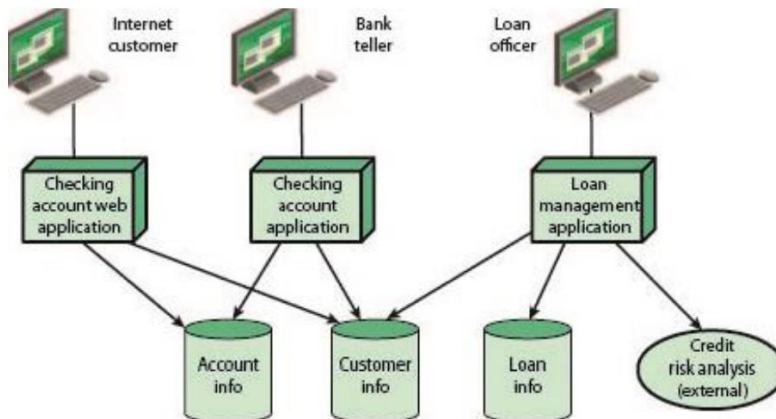
Service-Oriented Architecture (SOA): A form of client/server architecture used in enterprise systems. Organizes business functions into a modular structure rather than as monolithic applications for each department; as a result, common functions can be used by different departments internally and by external business partners as well.

Consists of a set of services and a set of client applications that use these services.

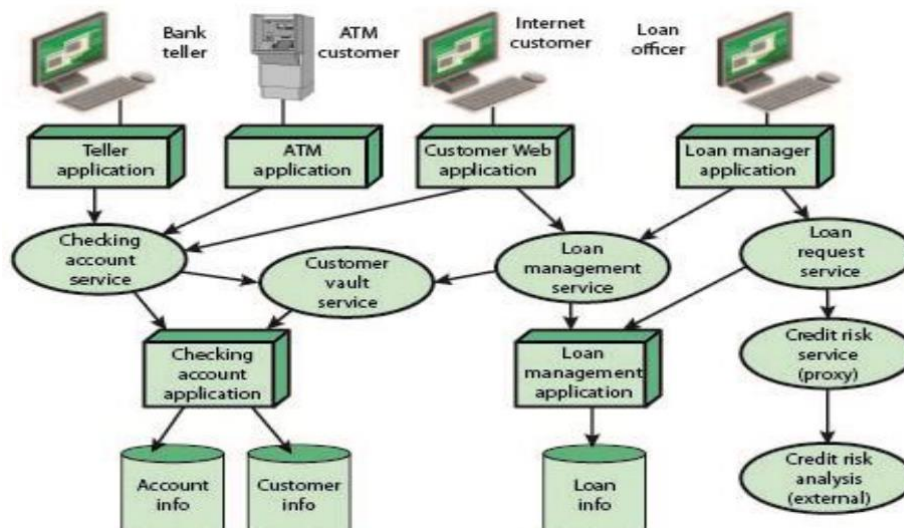
Standardized interfaces are used to enable service modules to communicate with one another and to enable client applications to communicate with service modules.



es. applicazione bancaria



es.



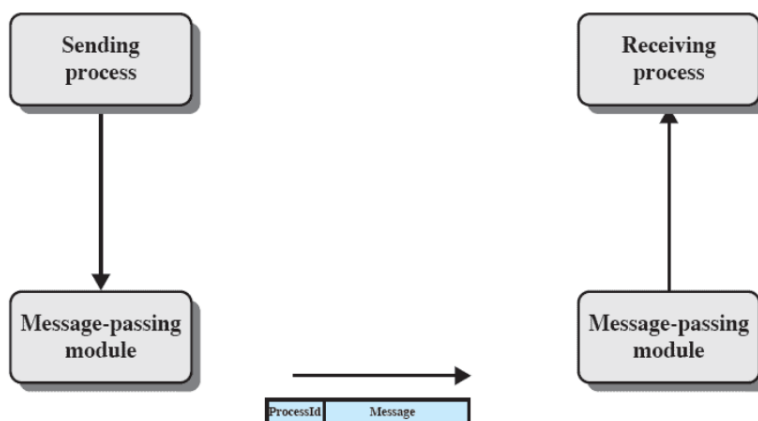
3. MESSAGE PASSING

Una parte del middleware sarà orientata allo scambio di messaggi.

Distributed message passing:



Primitive di base per il message passing:



4. REMOTE PROCEDURE CALLS

Remote procedure calls allow programs on different machines to interact using simple procedure call/return semantics. Used for access to remote services. Widely accepted and common method for encapsulating communication in a distributed system.

RPC is standard technology.

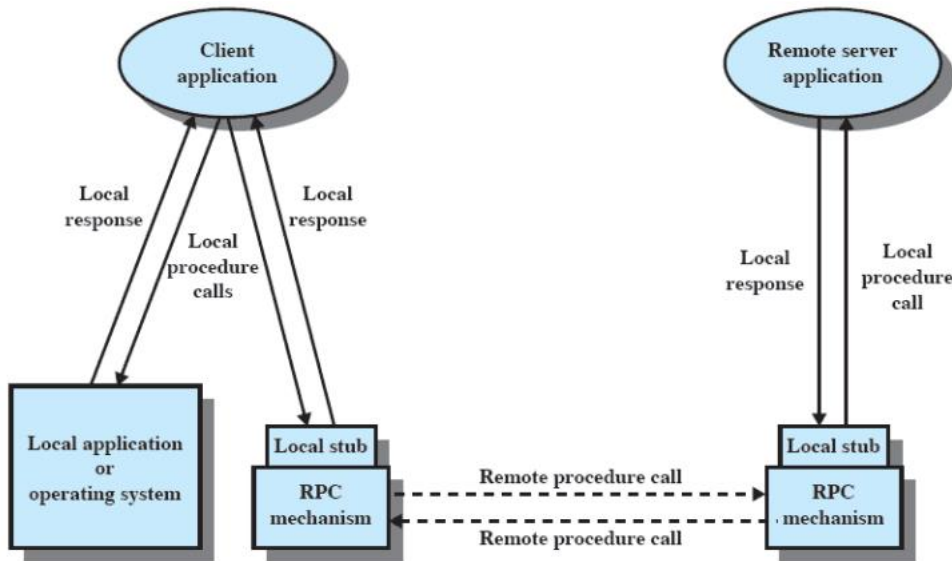
Advantages of standardization:

- the communication code for an application can be generated automatically
- client and server modules can be moved among computers and OSs with little modification and recoding

Remote Procedure Call Architecture:



Remote Procedure Call Mechanism:



Passing a parameter by value is easy; passing by reference is more difficult: a unique system wide pointer is necessary, the overhead for this capability may not be worth the effort.

The representation/format of the parameter and message may be difficult if the programming languages differ between client and server.

Client/Server Binding: a binding is formed when two applications have made a logical connection and are prepared to exchange commands and data.

Nonpersistent Binding: nonpersistent binding means that a logical connection is established between the two processes at the time of the remote procedure call and that as soon as the values are returned, the connection is dismantled.

The overhead involved in establishing connections makes nonpersistent binding inappropriate for remote procedures that are called frequently by the same caller.

Persistent Binding: a connection that is set up for a remote procedure call is sustained after the procedure return. The connection can then be used for future remote procedure calls. If a specified period of time passes with no activity on the connection, then the connection is terminated.

For applications that make many repeated calls to remote procedures, persistent binding maintains the logical connection and allows a sequence of calls and returns to use the same connection.

Synchronous RPC: behaves much like a subroutine call, behavior is predictable, however, it fails to exploit fully the parallelism inherent in distributed applications. This limits the kind of interaction the distributed application can have, resulting in lower performance.

Asynchronous RPC: does not block the caller, replies can be received as and when they are needed, allow client execution to proceed locally in parallel with server invocation.

Normal RPC functioning may get disrupted (call or response message is lost, caller node crashes and is restarted, callee node crashes and is restarted).

The call semantics determines how often the remote procedure may be executed under fault conditions.

Vediamo le diverse semantiche:

- **At least once:** this semantics guarantees that the call is executed one or more times but does not specify which results are returned to the caller.

Very little overhead & easy to implement using timeout based retransmission without considering orphan calls (i.e., calls on server machines that have crashed). The client machine continues to send call requests to the server machine until it gets an acknowledgement. If one or more acknowledgements are lost, the server may execute the call multiple times.

Works only for idempotent operations.

- **At most once:** guarantees that the RPC call is executed at most once. Either it does not execute at all or it executes exactly once depending on whether the server machine goes down.

Unlike the previous semantics, this semantics require the detection of duplicate packets, but works for non-idempotent operations.

- **Exactly once:** the RPC system guarantees the local call semantics assuming that a server machine that crashes will eventually restart.

It keeps track of orphan calls and allows them to later be adopted by a new server. Requires a very complex implementation.

5. CLUSTERS

Alternative to symmetric multiprocessing (SMP) as an approach to providing high performance and high availability.

Group of interconnected, whole computers (system that can run on its own, apart from the cluster) working together as a unified computing resource that can create the illusion of being one machine.

Each computer in a cluster is called a **node**.

Benefits of Clusters:

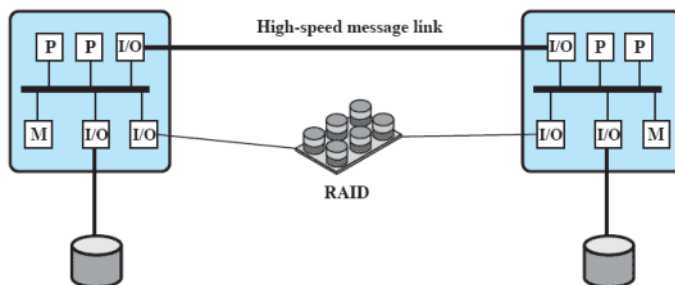
- Absolute scalability, it is possible to create large clusters that far surpass the power of even the largest stand-alone machines
- Incremental scalability, configured in such a way that it is possible to add new systems to the cluster in small increments
- High availability, failure of one node is not critical to system
- Superior price/performance, by using commodity building blocks, it is possible to put together a cluster at a much lower cost than a single large machine

Cluster configurations:

Standby server with no shared disk:



Shared disk:



Clustering Methods:

- **Passive Standby:** a secondary server takes over in case of primary server failure.

Benefits:

Easy to implement.

Limitations:

High cost because the secondary server is unavailable for other processing tasks.

- **Active Secondary:** the secondary server is also used for processing tasks.

Benefits:

Reduced cost because secondary servers can be used for processing.

Limitations:

Increased complexity.

- **Separate Servers:** separate servers have their own disks. Data is continuously copied from primary to secondary server.

Benefits:

High availability.

Limitations:

High network and server overhead due to copying operations.

- **Servers Connected to Disks:** servers are cabled to the same disks, but each server owns its disks. If one server fails, its disks are taken over by the other server.

Benefits:

Reduced network and server overhead due to elimination of copying operations.

Limitations:

Usually requires disk mirroring or RAID technology to compensate for risk of disk failure.

- **Servers Share Disks:** multiple servers simultaneously share access to disks.

Benefits:

Low network and server overhead. Reduced risk of downtime caused by disk failure.

Limitations:

Requires lock manager software. Usually used with disk mirroring or RAID technology.

Failure management: two approaches can be taken to deal with failures:

- **Highly available clusters:**
Offers a high probability that all resources will be in service.
Any lost query, if retried, will be serviced by a different computer in the cluster.
The cluster operating system makes no guarantee about the state of partially executed transactions.
If a failure occurs, the queries in progress are lost.
- **Fault-tolerant clusters:**
Achieved by the use of redundant shared disks and mechanisms for backing out uncommitted transactions and committing completed transactions.
Ensures that all resources are always available.

Fallover: the function of switching an application and data resources over from a failed system to an alternative system in the cluster.

Fallback: the restoration of applications and data resources to the original system once it has been fixed.

Fallback can be automated but this is desirable only if the problem is truly fixed and unlikely to recur. Automatic fallback can cause subsequently failed resources to bounce back and forth between computers, resulting in performance and recovery problems.

A cluster requires an effective capability for balancing the load among available computers. This includes the requirement that the cluster be incrementally scalable.

When a new computer is added to the cluster, the load-balancing facility should automatically include this computer in scheduling applications.

Middleware must recognize that services can appear on different members of the cluster and may migrate from one member to another.

Parallelizing Computation:

Parallelizing compiler:

- determines, at compile time, which parts of an application can be executed in parallel
- performance depends on the nature of the problem and how well the compiler is designed

Parallelized application:

- the programmer designs the application to run on a cluster and uses message passing to move data, as required, between cluster nodes
- this places a high burden on the programmer but may be the best approach for exploiting clusters for some applications

Parametric computing:

- this approach can be used if an application is an algorithm or program that must be executed a large number of times, each time with a different set of starting conditions or parameters
- for this approach to be effective, parametric processing tools are needed to organize, run, and manage the jobs in an orderly manner

Cluster Computer Architecture:

